# 이재영

백엔드 개발자

## 원인을 규명하고 결과를 재측정하는 백엔드 개발자

경기대학교 컴퓨터공학부를 졸업(2025.02)하고, 삼성청년SW·AI아카데미를 수료(2025.07~2026.06)했습니다. 팀 프로젝트에서 분석·집계와 발송 결과 적재 모듈을 맡았습니다. 메시지 단건 INSERT 구조를 배치 적재로 바꾸고 원인이 된 PK 생성 전략까지 규명했습니다. 설계 과정에서 선택지와 기각 사유를 ADR로 남겼습니다. 팀 ADR 19건 중 12건을 작성했습니다. 프로젝트가 끝난 뒤에는 제가 만든 구간의 성능을 다시 측정해 성립하는 조건의 범위를 함께 정리했습니다.

GitHub github.com/ljy0221
Email woduddl1000@gmail.com
Portfolio 2026.10

# 기술 요약

**주 스택**: Java 21 · Spring Boot 3.5 · Spring Data JPA · Flyway · MySQL 8 · Redis · Apache Kafka · Docker / Testcontainers · GitLab CI

### 설계·검증 관점에서 설명할 수 있는 것

| 축 | 내용 | 근거 시점 |
|---|---|---|
| 설계 결정 기록 | 선택지·기각 사유·그 결정이 남기는 위험을 ADR로 기록 | 프로젝트 기간 |
| 장애 원인의 경계 분리 | 프록시 자기 호출 / 트랜잭션 경계 / 컨테이너 한도와 JVM 힙 / 파이프라인 단계별 날짜 정의 | 프로젝트 기간 |
| 인덱스와 실행계획 | 인덱스가 "있는데 안 타는" 두 가지 사례: 조건절과 선두 컬럼 불일치 / SRID 속성 누락 | 프로젝트 종료 후 재현 실험 |

## 프로젝트

| # | 프로젝트 | 기간 | 한 줄 | 대표 성과 |
|---|---|---|---|---|
| 1 | **Butterfly CRM** | 2026.04 ~ 2026.05 | 이벤트 기반 고객 메시징·발송 분석 플랫폼 | 적재 91.8→1,485.3건/초 · ADR 12건 |
| 2 | **AiRadar** | 2026.02 ~ 2026.03 | 논문·뉴스 수집·분석 및 추천 서비스 | 캐시 히트 서버 처리 p50 0.45ms |
| 3 | **SYNAPSE** | 2026.01 ~ 2026.02 | 실시간 협업 코드 실행 노트 | 실행 지연 약 2.4배 단축(세션 재사용) |
| 4 | **갈래말래** | 2025.11 ~ 2025.12 | 여유 시간 기반 놀거리 추천 | 반경 5km 검색 약 13배(재현 실험) |
| 5 | **마음씨** | 2024.02 ~ 2024.08 | 우울증 관리 챗봇 및 일상관리 앱 | 캡스톤 경진대회 대상 |

## 표기 기준

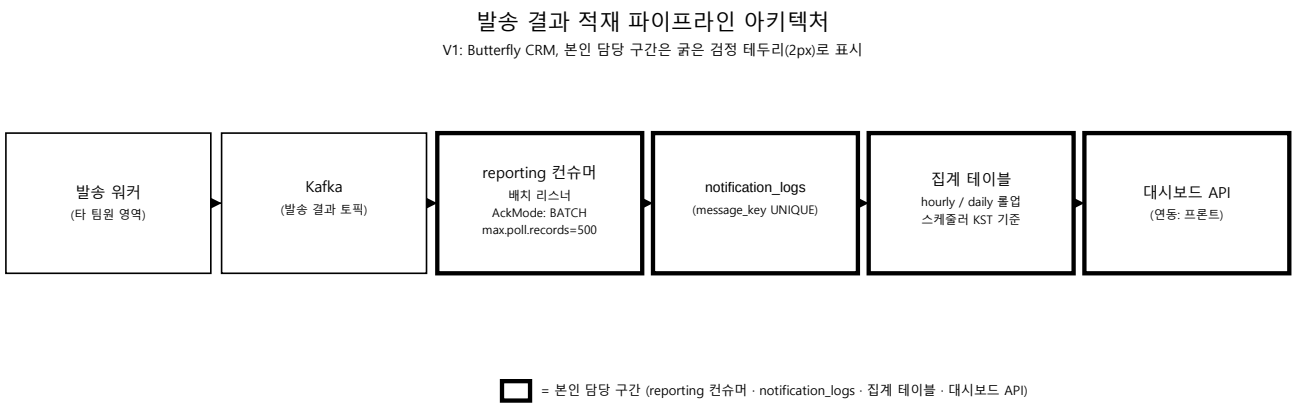
시각 자료의 색은 빨강 = 문제 상태·개선 전, 초록 = 해결 상태·개선 후, 검정 = 구조·흐름·모든 텍스트, 굵은 검정 테두리 = 본인 담당 구간입니다.

# 1. Butterfly CRM

발송 결과 적재와 집계 파이프라인

| | |
|---|---|
| **기간** | 2026.04 ~ 2026.05 (6주) |
| **팀** | 5명 (백엔드 3 · 인프라 1 · 프론트 1): 백엔드 3명 중 1명 |
| **담당** | 집계·통계·분석 모듈(`reporting`): 분석·집계 파이프라인, 발송 결과 적재·추적<br>관측성 메인: LGTM 관측 서버(docker-compose), Grafana 대시보드·Loki 로그 패널·Prometheus scrape, MDC 기반 JSON 로그<br>CI/CD: GitLab CI 파이프라인, Jib 이미지 빌드 → GHCR 푸시까지 |
| **연동** | 발송 워커(Kafka 페이로드 스키마 합의) · 프론트(대시보드 API) · 인프라(GHCR 푸시 이후의 배포 자동화는 팀원 담당) |
| **스택** | `Java 21 · Spring Boot 3.5.13 · Spring Data JPA · Flyway · MySQL 8 · Kafka · Redis · Prometheus / Grafana / Loki · Docker / Testcontainers · GitLab CI` |
| **레포** | `github.com/ljy0221/butterfly_crm` |

캠페인 메시지를 채널별로 발송하고 그 결과를 집계해 대시보드로 보여 주는 서비스입니다. 저는 발송 워커가 보낸 결과 메시지를 Kafka로 받아 `notification_logs`에 적재하고 그 위에서 시간별·일별 집계를 만들어 대시보드 API로 내보내는 구간을 맡았습니다. 이 구간은 앞쪽과 뒤쪽의 성질이 다릅니다. 앞쪽은 재전달이 있는 **쓰기 처리량** 문제이고, 뒤쪽은 데이터가 쌓일수록 무거워지는 **집계 읽기** 문제입니다.




발송 결과 적재 파이프라인. 굵은 검정 테두리가 본인 담당 구간입니다.

## 1-1. Kafka 발송 결과 적재의 배치 전환

**S. 상황**: 캠페인 한 건을 발송하면 채널별 결과 메시지가 Kafka로 한꺼번에 들어옵니다. 그런데 이 결과를 받는 컨슈머가 메시지 1건마다 `notification_logs`에 INSERT하고 커밋했습니다. `hibernate.jdbc.batch_size=1000`을 설정해 두었는데도 실제로 실행되는 SQL은 단건 INSERT였습니다. 적재가 밀리면 그 위에 얹힌 시간별 집계와 대시보드가 함께 늦어집니다. 거기에 중복 적재를 막으려고 건마다 Redis SETNX 가드를 한 번 더 왕복하고 있었습니다.

**T. 과제**: 적재 처리량을 올리되 같은 `message_key`가 두 번 들어오지 않도록 하는 멱등성은 그대로 유지해야 했습니다. 재전달이 일어나는 구간이라 멱등성을 포기하는 선택지는 없었습니다.

**A. 행동: 기술 선택과 트레이드오프**

① **원인**: 배치 적재로 바꾸는 작업에 착수한 뒤에 원인을 규명했습니다. `NotificationLog`의 PK 생성 전략이 `IDENTITY`라 Hibernate가 PK 값을 받으려고 INSERT를 즉시 실행합니다. 그 때문에 배치 설정이 조용히 무효가 됩니다.

② **선택한 경로**: PK 채번을 거치지 않아야 배치가 실제로 묶이므로 JPA를 우회해 `NamedParameterJdbcTemplate.batchUpdate` + `INSERT IGNORE`로 적재했습니다. 단건 리스너는 배치 리스너(`AckMode` RECORD → BATCH, `max.poll.records=500`)로 바꿨습니다.

③ **멱등성 이관 근거**: Redis SETNX 가드 대신 `message_key` UNIQUE 제약으로 옮겼습니다. 발송 워커 담당 팀원과 Kafka 페이로드 스키마를 합의하면서 `message_key`를 멱등성 키로 쓰기로 정한 것이 그 근거였습니다. 그 결정이 `INSERT IGNORE` 기반 중복 방지로 이어졌습니다.

④ **대가**: `INSERT IGNORE`는 중복 외의 오류(길이 초과 등)도 경고로 삼킵니다. 배치 도중 예외가 나면 배치 전체가 롤백·재전달됩니다.

**R. 결과**: 적재 처리량 평균 **91.8건/초 → 1,485.3건/초** (3회 측정, 로컬 EmbeddedKafka + Testcontainers MySQL 8.0, 측정 조건은 아래 표에 병기). 이 값은 프로젝트 당시 적용한 개선을 대상으로, 개선 전후 두 커밋을 각각 체크아웃해 2026년 9월에 다시 측정한 값입니다.

**단건 적재 vs 배치 적재 처리량 비교**

| 구분 | 평균 처리량 | 3회 측정 범위 |
|---|---|---|
| 단건 INSERT (개선 전) | 91.8건/초 | 61.6 ~ 147.2 |
| 배치 적재 (개선 후) | 1,485.3건/초 | 1,413.9 ~ 1,527.1 |

유니크 messageKey 10,000건, 로컬 EmbeddedKafka(파티션 1·컨슈머 1) + Testcontainers MySQL 8.0, 3회. 개선 전 147.2는 원인 미확인 이상치. 로컬 상대 비교이며 배포 환경 절대값이 아님. 개선 전 값에는 이번 개선으로 제거한 Redis 가드 왕복 포함. 왼쪽 세로선: 빨강은 개선 전, 초록은 개선 후.

**I. 배운 점**: 설정을 넣는 것과 그 설정이 실제로 적용되는 것은 다릅니다. ORM의 기본 전략(IDENTITY)이 성능 옵션(`batch_size`)을 조용히 무효화할 수 있습니다. 무효화됐다는 신호는 로그에도 예외에도 남지 않습니다. 설정 파일만 보면 배치가 켜져 있는 것으로 읽히기 때문에 실행되는 SQL을 확인하기 전까지는 적용됐다고 볼 근거가 없습니다.

이 개선은 처리량을 올린 대신 **오류 처리 경로를 한 단계 성기게** 만들었습니다. 단건 INSERT일 때는 어떤 메시지가 왜 실패했는지 예외로 드러났지만 `INSERT IGNORE` 배치에서는 중복이 아닌 실패도 경고로 흘러가고 배치 전체가 재전달됩니다. 그래서 이 구조는 **처리량과 실패 관측성을 맞바꾸는 관계**입니다. 그 균형을 지키려면 배치 리스너의 롤백·재전달 동작을 테스트로 고정해 두어야 합니다.

## 1-2. ADR 작성과 Flyway 하이브리드 도입(ADR-0013)

팀 ADR 19건 중 12건을 최초 작성했습니다 (`backend/docs/adr/` 최초 추가 커밋 작성자 기준. ADR-0017은 공동 decision-maker).

대표 1건은 **ADR-0013(Flyway 도입과 번호 컨벤션)**입니다. 전 모듈이 `ddl-auto=update`라 DROP·RENAME이 누락되고 환경 간 스키마가 어긋나는 상태였습니다. reporting은 `audit_logs`의 append-only 트리거와 권한 분리가 필요해 JPA로 표현할 수 없었습니다. 번호 체계는 역할별 블록(FK 순서가 어긋남)과 모듈별 history 분리를 두고 비교한 끝에 개발 순서 V1, V2 방식을 골랐습니다. 전면 전환 대신 **reporting만 Flyway를 쓰는 하이브리드**로 정했습니다. ADR에 적은 이유는 "다른 모듈은 진행 중인 작업이 많아 일괄 전환은 위험"이라는 것이었습니다.

집계 테이블 설계(**ADR-0007**)도 같은 방식으로 남겼습니다. 기존 3종으로는 회차 분리·템플릿별 성과·실패 원인·전사 대시보드 요구를 받을 수 없었습니다. 회차를 집계 테이블의 PK에 넣는 안은 카디널리티가 폭증하고, 속성을 한 테이블에 통합하는 안은 인덱스 설계가 불가능했습니다. 집계 5종으로 나누되 회차는 PK가 아니라 실행 식별자와 시간별 윈도우 조회로 받는 쪽을 골랐습니다. 대가는 테이블 5종의 운영 복잡도입니다.

**I**: 기각한 선택지의 사유까지 남기면 같은 논의를 반복하지 않는다는 것을 ADR 작업에서 배웠습니다. 다만 하이브리드로 두면 스키마를 바꾸는 경로가 모듈마다 달라집니다. 실제로 엔티티·Flyway 마이그레이션·벌크 INSERT SQL 세 곳이 어긋난 지점이 있었습니다. 프로젝트 기간에 들어간 벌크 INSERT가 엔티티에도 Flyway V1~V21에도 없는 `updated_at` 컬럼에 값을 넣고 있었습니다(발견 시점과 수정은 아래 후속 블록). 그래서 ADR에는 결정이 남기는 위험과 그 위험을 확인할 수단을 함께 적습니다.

## 1-3. 관측성 · 집계 워커 · 통계 화면 · CI 게이트 · 집계 쿼리

- **관측성 인프라 구축**: LGTM 관측 서버를 docker-compose로 구성해 메인으로 맡았습니다. Grafana 대시보드 3종(Kafka 컨슈머 랙·배치 처리량 패널을 포함한 Worker 모니터링 등)과 Loki 로그 패널을 연결하고 Prometheus scrape를 설정했으며, 로그는 MDC 기반 JSON 구조화 로그로 남겨 장애 발생 시 추적 근거로 삼았습니다.
- **집계 워커 안정화**: hourly/daily 집계 워커가 `TransactionRequiredException`으로 간헐 실패하던 원인이 `this.aggregate()` 자기 호출이 CGLIB 프록시를 우회해 `@Transactional`이 걸리지 않는 것이었습니다. `@Lazy` self-bean 주입으로 프록시를 경유하게 하고, 날짜별 독립 트랜잭션으로 나눴습니다. 스케줄러 시간 기준은 KST로 고정했습니다. 전날 1일 집계는 D-7~D-1 재집계로 바꿔 지연 이벤트와 DLQ 재처리를 흡수하게 했습니다.
- **캠페인 통계 0 표시 차단**: KPI가 항상 0으로 보이던 원인이 두 겹이었습니다. `@ConditionalOnProperty` 키가 yml에 없어 Stub 구현이 조용히 주입됐습니다. 실구현도 새벽 1시에 롤업되는 daily 테이블만 읽어 당일 발송분이 0이었습니다. 설정 키를 채우고 daily 우선 + `notification_logs` raw 카운트 폴백으로 바꿨습니다.
- **CI 게이트 복구와 파이프라인 정리**: `allow_failure`가 붙어 게이트 구실을 못 하던 job을 복원했습니다. 그 뒤 전체 pipeline이 10분 53초에 GREEN으로 통과하는 상태를 확인했습니다. 실제 회귀 2건(Flyway V7/V8 충돌, 애플리케이션

컨텍스트 로드 실패)을 막았습니다. Testcontainers reuse로 integration-test 실행 시간도 줄였습니다 (14분 02초 → 9분 56초, **1회 표본**이고 캐시 상태에 따른 편차가 커 순수 효과는 약 -25% 추정).

· **Code Freeze D-3 감사**: 분석 도메인 9개 마일스톤을 머지한 뒤 5축 감사를 돌려 BLOCKER 0, HIGH 1건(스케줄러 cron zone 미명시)을 1파일 `+5/-2` 수정으로 잠갔습니다 (감사 대상은 본인 담당 분석 도메인).

· **집계 쿼리**: 시간별 집계 쿼리가 풀스캔이던 구간에 복합 인덱스를 넣은 뒤 집계 지연이 **3,743ms → 2.8ms**였습니다 (103만 건, MySQL 8.x, 1시간 윈도우 집계, 운영·스테이징 DB에서 직접 측정, 애플리케이션 코드 수정 없음). 컬럼 구성을 범위 조건인 `created_at`을 선두에 두고 GROUP BY 컬럼 `campaign_id`, `channel`을 뒤에 붙이는 `(created_at, campaign_id, channel)`로 정한 것은 제 선택입니다. 다만 이 구간에는 사전 집계 테이블도 함께 놓여 있어 **2.8ms가 인덱스 쪽 효과인지 사전 집계 조회 쪽 효과인지는 지금의 재현 실험으로 분리하지 못했습니다.** 측정 원본을 남겨 두지 않아 아래 후속 블록에 재현 실험 값을 함께 적었습니다.

## 1-4. 통합 테스트 삭제와 트랜잭션 경계 복구

위 개선 뒤 배치 리스너를 수정하는 과정에서 Kafka 통합 테스트 2파일(473줄)과 프로덕션의 `@Transactional` 2줄이 같은 커밋에서 함께 빠졌습니다. 커밋에 적힌 사유는 "불안정한 Kafka 통합 테스트를 제거한다"였습니다. `@Transactional` 제거 근거는 "배치 `KafkaListener`에 `@Transactional`을 붙이면 AOP 프록시가 `List<>` 파라미터 매칭을 방해할 수 있다"는 **검증하지 않은 가정**이었습니다.

프로젝트 종료 직전에 들어간 커밋입니다. 회귀는 CI를 통과한 채 남았습니다. 기존 CI가 보는 축이 "빌드와 기능 테스트가 통과하는가"였기 때문입니다. 회귀를 실제로 발견하고 복원한 시점과 그 크기는 아래 후속 블록에 적었습니다.

**I**: 지금은 **테스트 삭제와 프로덕션 변경을 한 커밋에 넣지 않고** 커밋에 들어가는 변경 범위를 diff로 확인한 뒤에 올립니다. 미검증 가정으로 코드를 지우지 않습니다. 게이트를 만들 때 "무엇을 막는 게이트인지"를 먼저 정의합니다. 1-3의 CI 게이트 복구로 막을 수 있었던 것은 마이그레이션 충돌과 컨텍스트 로드 실패였습니다. 성능과 롤백 동작은 그 게이트가 보는 축이 아니었습니다.

**[ 프로젝트 종료 후 · 개인 작업 · 2026.09 ]**

**1-4의 회귀와 1-2의 `updated_at` 결함을 이 시기에 발견했습니다.** `@Transactional`이 빠진 상태에서는 처리량이 크게 떨어졌고, 회귀를 발견한 뒤 `@Transactional`을 복원했습니다. `updated_at` 결함은 빈 MySQL에 Flyway V1~V21을 적용한 뒤 같은 INSERT로 `ERROR 1054`를 재현해 확인하고 컬럼 정의를 맞춰 고쳤습니다.

**집계 쿼리는 레포의 스키마와 쿼리를 그대로 두고, 행 폭·버퍼 풀은 제가 가정해 구성한 조건에서 다시 측정했습니다.** 그 조건에서는 **2,663ms → 3.26ms** (합성 103만 행, 행 폭 평균 1.1KB, MySQL 8.0 buffer pool 128MB, O_DIRECT, 1시간 윈도우, warm 5회)였습니다. 배율은 행 폭·버퍼 풀·시간 범위에 따라 크게 흔들렸고, 시간 범위를 7일 이상으로 넓히면 옵티마이저가 인덱스를 무시해 이득이 사라졌습니다. **이 실험에서 쓴 인덱스는 레포에 반영하지 않았습니다**(1-3의 당시 개선과는 별개의 실험용 인덱스입니다).

레포에 실제로 있던 보조 인덱스 7개는 이 쿼리의 조건절과 선두 컬럼이 맞지 않아 옵티마이저가 하나도 쓰지 않았습니다(EXPLAIN `type=ALL`). 인덱스는 "있다/없다"가 아니라 쿼리의 조건절과 선두 컬럼이 맞는가의 문제라는 것을 여기서 확인했습니다. 재측정 원본 출력은 모두 따로 보관했습니다.

집계 쿼리 실행계획 변화: EXPLAIN 발췌 비교

인덱스 없음 (개선 전)

```
type=ALL / key=NULL / rows=943334 / Extra=Using where; Using temporary
```

인덱스 적용 (개선 후)

```
type=range / key=b3c_ca_camp_chan (created_at, campaign_id, channel)
rows=1431 / Extra=Using index condition; Using temporary
```

※ "Using temporary"는 인덱스 적용 후에도 위·아래 양쪽에 그대로 남습니다: 인덱스로 사라지는 항목이 아닙니다.

같은 쿼리, 인덱스 DDL만 추가. 합성 103만 행 / 행 폭 평균 1.1KB / MySQL 8.0 buffer pool 128MB / O_DIRECT / 1시간 윈도우.
원본 환경을 재현한 것이 아니라 가정해 구성한 조건.

같은 쿼리, 인덱스 DDL만 추가. 합성 103만 행 / 행 폭 평균 1.1KB / MySQL 8.0 buffer pool 128MB / O_DIRECT / 1시간 윈도우. 원본 환경을 재현한 것이 아니라 가정해 구성한 조건.

**[ 프로젝트 종료 후 · 개인 작업 · 2026.10 ]**

삭제했던 Kafka 컨슈머 통합 테스트를 다시 만들었습니다. 지울 때 문제였던 EmbeddedKafka 대신 실제 브로커(Testcontainers Kafka)와 MySQL을 쓰고, 정상 적재, 같은 `message_key` 재전달 시 1행, blank 키 스킵, 배치 도중 예외 시 전체 롤백의 4개 케이스를 고정했습니다. 삭제된 473줄을 그대로 되살린 것이 아니라 필요한 동작만 새로 썼고, DLQ 라우팅·재시도와 Flyway 스키마 일치는 포함하지 않았습니다(이 모듈의 테스트는 `ddl-auto`로 스키마를 만듭니다).

이 테스트가 실제로 회귀를 잡는지 확인하려고 프로덕션 코드를 일부러 망가뜨려 봤습니다. `@Transactional`을 빼면 롤백 케이스가 실패했고, `updated_at`을 되살리면 4개 케이스가 모두 실패했습니다. 또 1-4에서 "AOP 프록시가 `List<>` 파라미터 매칭을 방해할 수 있다"고 가정했던 부분은 틀렸고, 프록시가 걸린 상태에서도 배치 리스너가 정상 동작했습니다.

# 2. AiRadar

추천 서빙 API와 수집 파이프라인

| | |
|---|---|
| **기간** | 2026.02 ~ 2026.03 (6주, 실 개발 약 4주) |
| **팀** | 6명 (백엔드 3 · 프론트 2 · 인프라 1) |
| **담당** | 추천 서빙 API(캐시·재랭킹·Cold Start)와 그 앞단인 Spark 메달리온 파이프라인·Airflow DAG, AI 서버(분석·임베딩 본체) |
| **연동** | 크롤링(수집 스키마) · 프론트(피드 API): 크롤링·Trend 집계·프론트와 AI 서버의 직업 위험도 모듈은 다른 팀원 영역이고, 컨테이너·실행 환경 설정은 인프라 담당 팀원과 함께 다뤘습니다 |
| **스택** | `Java 17 · Spring Boot 3.3.5 · Python · FastAPI · Kafka · Spark 3.5 · Airflow 2.8 · Delta Lake 3.1 · PostgreSQL 16 + pgvector · Redis` |
| **레포** | `github.com/ljy0221/AiRadar` |

## 2-1. 파이프라인 단계별 날짜 기준 통일

Bronze의 `batch_date`가 소스마다 달랐습니다(뉴스는 크롤링일, 논문은 `published_at`). Silver Job이 받는 `--date`는 Airflow `ds`여서 대상 파티션을 찾지 못했습니다. 그 결과는 실패가 아니라 **Silver 처리 0건**이었습니다. `batch_date`를 `to_date(kafka_timestamp)`(수신 시각) 기준으로 통일했습니다. 발행일 기준은 지연 도착분이 과거 파티션으로 흩어지고 DAG 실행일 기준은 수집 시각과 어긋납니다. 단계 간에 하나로 맞출 수 있는 값이 수신 시각이었습니다. 대신 지연 도착 중복을 받아들이고 anti-join 재처리를 함께 뒀습니다.

단계마다 "날짜"의 정의가 다르면 실패가 아니라 0건으로 끝나 발견이 늦어집니다. Silver의 기본 `--date`는 여전히 `ds`이고 타임존 설정이 레포에 없어 자정 부근 불일치 가능성은 남아 있습니다.

## 2-2. Airflow 컨테이너 OOM 원인 분석

단일 서버의 Airflow 컨테이너가 SparkSubmit driver를 직접 띄우는 구조(deploy-mode client)에서 컨테이너 메모리 한도를 넘겼습니다. 가장 유력한 원인 후보를 추려 순서대로 검증했습니다. 커밋 순서가 검증 순서와 일치합니다: ① 동시 실행 제한 → ② executor 축소 → ③ driver가 executor와 달리 Airflow 컨테이너 **안에서** 돈다는 점에 따라 컨테이너 `mem_limit` 2g → 6g (커밋 diff로 확인된 설정 변경 사실이며 메모리 실측 로그는 없음). 이 과정에서 docker-compose 구성과 Spark·Airflow 컨테이너 설정을 인프라 담당 팀원과 함께 조정했습니다.

이 상향 뒤에도 프로젝트 후반 ALS DAG에서 OOM이 다시 났습니다. driver 메모리를 줄이는 쪽으로 한 번 더 조정했습니다. 컨테이너 한도만으로 끝나는 문제가 아니었습니다.

**I**: OOM은 컨테이너 한도 / JVM 힙 / 호스트 중 어느 경계에서 났는지 먼저 나눠야 한다는 것을 이 과정에서 확인했습니다. 그 구분 없이 executor부터 줄인 것이 ②였습니다.

**[ 프로젝트 종료 후 · 개인 작업 · 2026.09 ]**

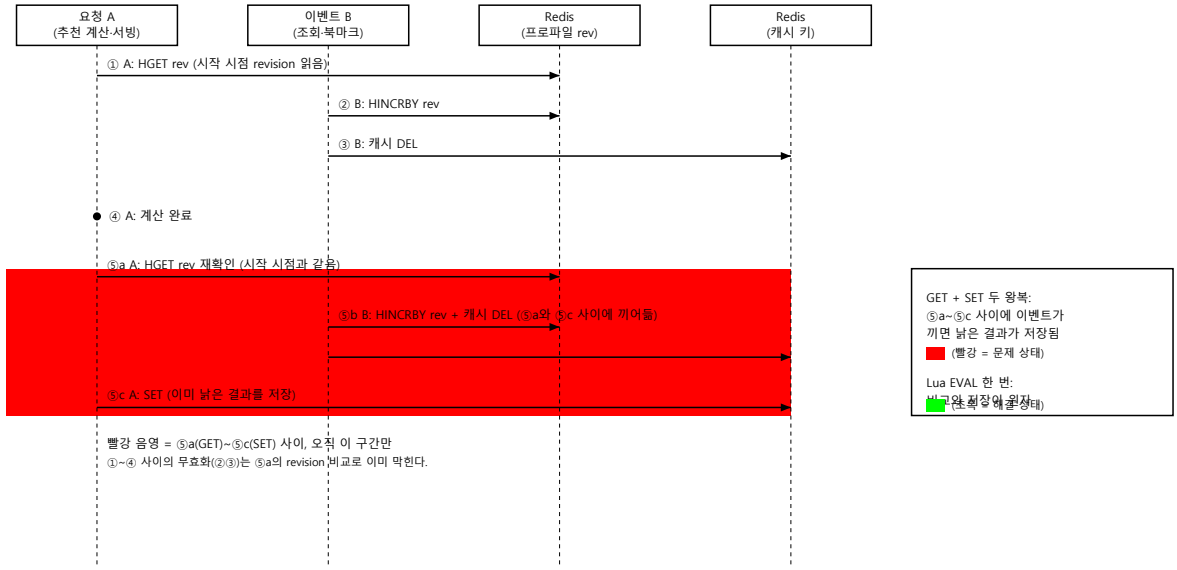
추천 캐시의 일관성 결함을 순서대로 고쳤습니다. 조회·북마크 뒤 캐시 무효화 누락과, 캐시 삭제 실패가 뒤의 처리까지 막던 문제를 먼저 분리했고, 남은 것은 계산 도중 캐시가 무효화되면 오래된 결과가 다시 저장되는 경합이었습니다. 프로파일 rev를 HINCRBY로 먼저 올린 뒤 캐시를 지우고 서빙이 저장 직전에 rev를 비교하게 했는데, **확인(GET)과 저장(SET)이 별개 왕복이라 그 사이가 그대로 경합 구간으로 남았습니다.** 그래서 HGET rev 비교와 SET PX를 Lua 스크립트 하나로 묶어 원자화했습니다(단일 Redis 노드 전제라 Cluster에서는 해시 태그가 필요합니다).

검증은 실제 Redis에 붙는 통합 테스트 10개로 했습니다 (Redis만 실물이고 PostgreSQL·리포지토리는 mock, 환경변수가 없으면 skip되는 조건부 테스트). 응답 시간이나 히트율 개선은 주장하지 않습니다. 다중 프로세스 수준의 경합은 **아직 재현하지 못했습니다.**

같은 기간에 추천 API 응답 시간의 성립 조건을 좁히려고 다시 측정했습니다. 캐시 히트는 서버 처리 p50 0.45ms, 캐시 미스는 클라이언트 p50 5.14ms / p95 7.06ms (로컬 Docker, 합성 데이터, 3회 × 2,000요청 중앙값, 동시성 1)였고, "50ms 이하"는 **동시성 10 이하에서만** 성립했습니다. 신규 사용자 폴백 경로는 결과를 캐시에 저장하지 않아 미스 경로보다 느렸습니다 (동시성 1 클라이언트 p50 약 13ms).

같은 실험에서 Redis를 완전히 정지시킨 상태도 시험했습니다. 이 상태에서는 추천 API가 클라이언트 타임아웃에 도달할 때까지 응답을 돌려주지 않았습니다(3회 호출 3회 모두 동일, 원인 미확인). Redis 장애를 받아내는 폴백 경로는 이번 재현 실험으로 확인한 추가 구현 과제입니다.

재캐싱 경합과 Lua 원자화 시퀀스




코드상 순서는 revision 증가(HINCRBY)가 캐시 삭제(DEL)보다 먼저다: UserEventService.invalidateCache. ②③과 ⑤b 모두 이 순서로 그렸다.

재캐싱 경합과 Lua 원자화 시퀀스. 빨강 음영은 rev 확인(GET)과 저장(SET) 사이의 경합 구간이며, Lua EVAL 한 번으로 비교와 저장을 원자화해 닫은 창입니다.

# 3. SYNAPSE

코드 실행 환경과 실시간 편집

| | |
|---|---|
| **기간** | 2026.01 ~ 2026.02 (6주) |
| **팀** | 6명 (백엔드 4 · 프론트 2): 팀장 |
| **담당** | 백엔드: 코드 블록 저장, DB 설계, 블록 이력(MongoDB), AI 코드 리뷰·요약<br>프론트: Electron IPC 연결, 코드 블록 구현, 코드 실행 세션 |
| **연동** | Yjs 실시간 동기화: 서버와 토큰 발급은 다른 팀원 영역이고, 본인은 클라이언트 측 수정을 맡았습니다 |
| **스택** | `Java 17 · Spring Boot 3.5.9 · PostgreSQL · MongoDB · Redis · Docker · Electron · TypeScript` |
| **레포** | `github.com/ljy0221/SynapseNote` |

## 3-1. 코드 실행 격리 단위 변경: 세션 단위 컨테이너 재사용

**S. 상황**: 사용자가 작성한 코드를 그대로 실행하는 기능이라 초기 설계의 전제는 "실행 단위로 완전히 격리해야 한다"였습니다. 블록을 실행할 때마다 임시 파일을 만들고 `docker run --rm`으로 새 컨테이너를 띄운 뒤 종료·삭제했습니다.

**T. 과제**: 그 격리 비용은 두 갈래로 돌아왔습니다. 실행마다 컨테이너 기동·정리 시간이 그대로 지연에 얹혔습니다. 동시에 뜨는 컨테이너 수만큼 리소스 부담도 커졌습니다.

**A. 행동: 기술 선택과 트레이드오프**: 격리 단위를 실행에서 세션으로 낮췄습니다. 처음에 완전 격리를 전제로 설계한 것도, 지연과 리소스 부담을 보고 세션 재사용으로 전환하기로 한 것도 제 판단입니다. 팀장으로서 이 전환을 말로 설득하는 대신 세션 재사용 프로토타입을 먼저 구현해 처리 속도 차이를 보여주고 팀의 동의를 얻었습니다. 세션 키를 `noteId:language`로 두고 `docker run -d -i`로 REPL 컨테이너를 유지한 뒤 `docker attach`로 stdin에 코드를 넣었습니다. 실행 종료 판정은 실행마다 고유한 endMarker를 코드 뒤에 출력시키고 50ms 폴링으로 검출하는 방식을 골랐습니다. 지연 하한이 생기는 대신 언어별 REPL 프롬프트 처리에 의존하지 않습니다. 출력이 뒤섞이지 않도록 세션별 실행 큐로 직렬화했습니다(긴 블록이 뒤를 막는 것이 대가). REPL 구조와 맞지 않는 Java는 단일 실행 모드로 폴백합니다. 가장 큰 대가는 기능 쪽입니다. 같은 세션 안에서는 블록 간 상태가 공유되므로 격리를 일부 포기합니다.

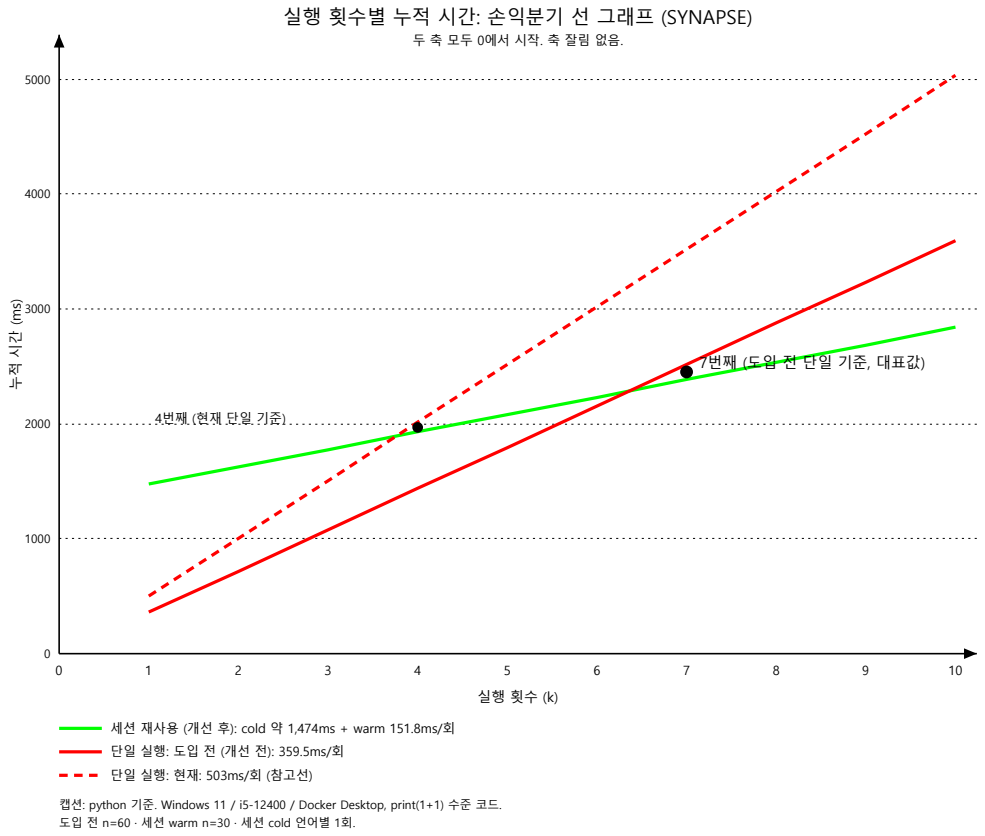
**R. 결과**: 두 번째 실행부터 컨테이너 기동·정리 비용이 사라졌습니다. 대신 세션을 처음 여는 실행에는 컨테이너 기동과 attach 후 고정 1초 대기가 코드상 그대로 남아 있어 이 결정의 이득은 한 노트에서 같은 언어를 여러 번 실행하는 패턴에서만 나옵니다. 수치는 프로젝트 종료 후 측정한 것이라 아래 후속 블록에 적었습니다.

**I. 배운 점**: 재사용 최적화에는 손익분기가 있습니다. "평균이 빨라졌다"가 아니라 "어떤 사용 패턴에서 이득인가"로 말해야 한다는 것이 이 작업의 결론입니다. 재작성 과정에서 유휴 정리 기준이 마지막 사용이 아니라 세션 생성 시점이 된 것으로 **코드상 보이지만 재현은 하지 않았습니다.**

**[ 프로젝트 종료 후 · 개인 작업 · 2026.09 ]**

도입 직전 커밋과 현재 코드를 각각 체크아웃해 다시 측정했습니다. 단일 실행 평균 python 359.5ms / javascript 385.1ms → 세션 warm 평균 151.8ms / 161.7ms로 **약 2.4배** (Windows 11 / Docker Desktop, `print(1+1)` 수준 코드라 컨테이너 오버헤드 하한값, 도입 전 n=60 · 세션 warm n=30). 세션 cold 첫 실행은 python·javascript 모두 **약 1.5초** (동일 환경, 언어별 1회뿐)로, 단일 실행보다 느리다는 것도 이때 수치로 확인했습니다.

이 기능은 주피터 노트북의 코드 셀처럼, 한 노트에서 같은 언어의 코드를 여러 번 실행하는 사용 패턴을 전제로 설계했습니다. 누적 시간으로 계산하면 **도입 전 단일 실행 기준으로 7번째 실행부터** 이득이 나기 시작해, 그 전제가 맞는 사용 패턴에서는 격리 단위를 낮춘 선택이 누적 시간을 줄입니다.




python 기준. Windows 11 / i5-12400 / Docker Desktop, `print(1+1)` 수준 코드. 도입 전 n=60 · 세션 warm n=30 · 세션 cold 언어별 1회. 두 축 모두 0에서 시작하며 축을 자르지 않았습니다.

## 3-2. Yjs 피드백 루프 차단과 블록 이력 모델링

팀이 구현한 Yjs 실시간 동기화에서 외부 값을 반영하면 그 반영이 다시 변경 이벤트를 일으켜 돌아오는 피드백 루프가 있었습니다. programmatic dispatch 구간에서 이 루프를 차단하도록 클라이언트 측을 수정했습니다. 블록 이력은 MongoDB 문서로 모델링해 (블록, 슬롯) 유니크 인덱스를 정의했습니다.

**[ 프로젝트 종료 후 · 개인 작업 · 2026.10 ]**

블록 저장소를 PostgreSQL + MongoDB에서 PostgreSQL 하나로 합쳐도 되는지 PoC로 확인했습니다. 저장소 접근을 인터페이스 뒤로 분리하고 PG JSONB 어댑터를 붙여, 같은 데이터와 시나리오로 세 구성을 비교했습니다. 현행 모델 그대로 실행 이력을 JSONB 배열에 두면 이력이 길어질수록 느려졌습니다. 이력 5,000건에서 조회 후 전체 저장은 PG 36.0ms로 Mongo 16.8ms의 **약 2.1배**, 부분 갱신은 25.6ms 대 4.3ms로 **약 6배**였습니다 (median, 시나리오 M 100노트×500블록). 이력을 별도 테이블로 분리한 구성은 이력 길이와 무관하게 1.3ms 안팎으로 일정했습니다. 노트 삭제 정합성은 Mongo를 중지시킨 장애 주입 10건 모두에서 PG에서는 노트가 지워지고 블록이 남는 불일치가 났고, PG 단일 트랜잭션은 예외 주입 10건이 전부 롤백되어 불일치가 0건이었습니다 (발생 빈도가 아닌 동작 시연).

결론은 "PG 하나로 가능, 단 이력 분리가 조건"입니다. 반대 결과도 있었습니다. 단건 속성 갱신은 노트당 5,000블록 시나리오에서 Mongo가 0.35ms, PG가 0.88~1.69ms(이력 분리 / JSONB 배열 구성)로 Mongo가 빨랐습니다(절대 차이 약 1ms). 5,000건은 극단적인 길이이고 500건 미만에서는 차이가 수 ms 이내였습니다. 현행 3DB에서 이력만 분리하는 비교군은 시험하지 않았고, 장애 시험도 3DB는 Mongo 컨테이너를 실제로 중지, 2DB는 예외 주입으로 방식이 달랐습니다. 수치는 Node 클라이언트와 Docker Desktop 환경에서 얻은 것이라 Java 서비스의 절대값이 아니고, 이력 분리 구성과 Yjs 서버의 영속 계층 이전은 구현하지 않고 벤치로만 확인했습니다. 이전을 결정하기 전에 Java 하네스로 재측정해야 합니다.

# 4. 갈래말래

공간 검색

| | |
|---|---|
| **기간** | 2025.11 ~ 2025.12 (WBS 기준) |
| **팀** | 2명 |
| **담당** | hotplace · review 도메인 / AI 서버(RAG 기반 장소 Q&A·소개문 생성) / 공공데이터 연동 / AI 서버 배포와 GitLab CI/CD |
| **연동** | 통합 서버 배포와 인증은 다른 팀원 영역입니다 |
| **스택** | `Java 17 · Spring Boot 3.4.12 · MyBatis · MySQL 8`(공간 함수) `· Python · FastAPI · AWS EC2 · GitLab CI/CD` |
| **레포** | `github.com/ljy0221/SpontaneousTrip` |

2인 팀에서 hotplace 도메인과 반경 검색을 담당했습니다. "남은 시간 안에 갈 수 있는 곳"을 레포 덤프 기준 약 5만 행 규모의 좌표 데이터에서 찾아야 했습니다 (전량 이관 시 hotplace 50,915행, 로컬 재현 기준). 검색 구조는 2단 필터로 정했습니다. 정밀 거리 계산을 전 행에 돌리는 비용을 줄이려고 교통수단별 속도로 반경을 정해 바운딩 박스를 만듭니다. 그 뒤 `MBRContains`로 후보를 좁히고 `ST_Distance_Sphere`로 정밀 거리를 거릅니다.

**[ 프로젝트 종료 후 · 개인 작업 · 2026.09 ]**

레포 쿼리를 그대로 두고 인덱스 조건만 바꿔 다시 측정했습니다. `pos POINT NOT NULL SRID 4326` + SPATIAL INDEX를 적용하면 반경 5km 검색이 **191.5ms → 14.7ms** (Docker MySQL 8.4.11, 50,915행, 4지점 × warm 5회 평균, 서버 측 측정)였습니다. 확인한 것은 **인덱스를 만드는 것과 인덱스를 타는 것은 다르다**는 점입니다: SPATIAL INDEX를 만들어도 컬럼에 SRID 4326 속성이 없으면 옵티마이저가 쓰지 않습니다(EXPLAIN `type=ALL`). 이 인덱스는 이후 레포에 DDL 스크립트(db/hotplace_spatial_index.sql)로 추가했고, 좌표 순서를 바로잡은 마이그레이션(db/Data_Migration_fixed_latlon.sql)도 함께 넣었습니다. 기존 스키마·마이그레이션 파일은 수정하지 않았고, 운영 DB에 실제로 적용했는지는 확인하지 못했습니다.

# 5. 마음씨

우울증 관리 챗봇 및 일상관리 앱

| | |
|---|---|
| **기간** | 2024.02 ~ 2024.06 (캡스톤), 이후 2024.07 ~ 2024.08 유지보수 |
| **팀** | 5명 (백엔드 1 · 프론트 2 · AI 2): 프론트 2명 중 1명 |
| **담당** | Flutter 프론트엔드: 일기 · 캘린더 · 감정 통계 · 우울 설문 · 채팅 화면, 로그인·회원가입을 포함한 Spring 서버 연동 / 유지보수: 배경음악 재생, 로그인 상태 전역 관리 |
| **연동** | Spring Boot(MyBatis)와 AI 서버 Flask(KoELECTRA)를 클라이언트에서 각각 호출했고, 행동 추천 화면은 다른 프론트 팀원 영역입니다 |
| **스택** | 캡스톤: `Dart 3.3` 이상 · `Syncfusion Charts` · `FlutterSecureStorage` / 유지보수: `Dart 3.4` 이상 · `Provider` · `fl_chart` |
| **레포** | `github.com/Chaeruin/RestProject` (팀 레포, 본인 작업 브랜치 LJY) |

Flutter·Dart 경험 없이 프론트엔드를 맡아 캡스톤 기간에 일기·감정 통계·우울 설문·채팅 화면과 Spring API 연동을 만들었고, 경진대회에서 대상을 받았습니다. 이후 같은 주제로 2024년 공개SW개발자대회에 출전했으나 예선에서 탈락했고, 캡스톤 기간에 미비했다고 판단한 부분을 개선하려고 유지보수를 이어가며 기술 선택을 바꿨습니다.

가장 크게 바꾼 것은 상태 관리입니다. 캡스톤 버전은 `memberId`를 보안 저장소에서 화면마다 읽거나 생성자로 넘겼는데(prop drilling), 화면이 깊어질수록 전달 경로가 길어져 코드가 지저분했고 제가 지향하는 개발 방식과도 맞지 않는다고 판단했습니다. 그래서 로그인 정보를 `AuthProvider`로 올려 `SharedPreferences`에 영속화하고 홈·채팅·일기·통계·추천·로그인 화면에 적용했으며, 배경음악은 `AudioProvider`로 분리해 화면을 옮겨도 이어 재생되게 했습니다. 통계 차트는 Syncfusion Charts가 정해진 차트 유형 위주여서 커스터마이징에 한계가 있다고 판단해 `fl_chart`로 바꿨습니다.

전환은 **부분 적용**에 그쳤습니다. 행동·일기 작성·통계 위젯·설문 화면에는 생성자 전달이 남아 있습니다. 전역으로 올릴 값과 화면에 남길 값의 경계를 먼저 정하지 않으면 두 방식이 섞인 채 남는다는 것이 이 프로젝트에서 남은 점입니다.

# 수상

**캡스톤디자인 산학협력 프로젝트 경진대회 대상** (2024.06, 경기대학교 · 마음씨, 우울증 관리 챗봇 및 일상관리 앱)

# 연락처

이름 이재영 · 직무 백엔드 개발자 · GitHub `github.com/ljy0221` · 이메일 `woduddl1000@gmail.com`